

Objectifs

À l'issue de cette section, l'étudiant devra être capable de :

- calculer le pgcd de deux entiers avec l'algorithme d'Euclide et déterminer des coefficients de Bézout avec l'algorithme d'Euclide étendu ;
- effectuer des calculs en arithmétique modulaire, notamment l'exponentiation rapide et le calcul d'un inverse modulo m ;
- expliquer et implémenter le crible d'Ératosthène pour déterminer les nombres premiers jusqu'à un entier N ;
- appliquer la technique du retour sur trace pour résoudre un problème combinatoire, en particulier le Sudoku.

1 Arithmétique

1.1 Algorithme d'Euclide

Le plus célèbre des algorithmes est très certainement l'algorithme d'Euclide. Il permet de calculer le plus grand commun diviseur de deux entiers (dit « pgcd », en anglais gcd pour *greatest common divisor*). Étant donnés deux entiers u et v positifs ou nuls, notons $\gcd(u, v)$ leur pgcd. On a les propriétés suivantes :

$$\gcd(u, 0) = u \quad (1)$$

$$\gcd(u, v) = \gcd(v, u) \quad (2)$$

$$\gcd(u, v) = \gcd(u - v, v) \quad \text{si } u \geq v \quad (3)$$

En particulier, il est d'usage, et pratique, de poser $\gcd(0, 0) = 0$. Des identités ci-dessus, on peut notamment déduire que

$$\gcd(u, v) = \gcd(v, u \bmod v) \quad (4)$$

en soustrayant $\lfloor u/v \rfloor$ fois v à u .

C'est cette dernière identité que met en œuvre l'algorithme d'Euclide, en la répétant jusqu'à ce que v soit nul. Il renvoie alors la valeur de u , qui est le pgcd des valeurs initiales de u et v .

Le code est immédiat ; il est donné dans le programme suivante.

Algorithme d'Euclide

On peut l'écrire récursivement (ici en OCaml)

```
let rec gcd (u: int) (v: int) : int =
  if v = 0 then u else gcd v (u mod v)
```

ou avec une boucle (ici en C)

```
int arith_gcd(int u, int v) {
  while (v != 0) {
    int tmp = v;
    v = u % v;
    u = tmp;
  }
  return u;
}
```

La terminaison de cet algorithme est assurée par la décroissance stricte de v et le fait que v reste par ailleurs positif ou nul. On a en effet l'invariant de boucle évident $u, v \geq 0$.

La correction de l'algorithme repose sur le fait que l'instruction 4 préserve le plus grand diviseur commun. Quand on parvient à $v = 0$, on renvoie alors u c'est-à-dire $\gcd(u, 0)$, qui est donc le pgcd des valeurs initiales de u et v .

La complexité de l'algorithme d'Euclide est donnée par le théorème de Lamé.

Théorème de Lamé

Soient deux entiers $u > v > 0$. Si l'algorithme d'Euclide effectue k itérations (c'est-à-dire k divisions) pour calculer le pgcd de u et v , alors $u \geq F_{k+2}$ et $v \geq F_{k+1}$ où (F_n) est la suite de Fibonacci. Cette borne est atteinte.

Démonstration. La preuve se fait par récurrence sur k .

Le cas $k = 0$ est exclu car il faut au moins une étape pour atteindre le cas de base $v = 0$. Pour $k = 1$, on a bien $u \geq 2 = F_3$ et $v \geq 1 = F_2$.

Pour $k > 1$, on effectue $k - 1$ itérations pour calculer le pgcd de v et $u \bmod v$. On a bien $v > u \bmod v > 0$, le cas $u \bmod v = 0$ étant exclu, sans quoi on aurait une seule division. Par hypothèse de récurrence, on a donc $v \geq F_{(k-1)+2} = F_{k+1}$ et $u \bmod v \geq F_{(k-1)+1} = F_k$. Or $u > v$ et donc $u \geq v + u \bmod v \geq F_{k+1} + F_k = F_{k+2}$.

Cette borne est atteinte : le calcul de $\gcd(F_{k+2}, F_{k+1})$ nécessite exactement k itérations. En effet, les valeurs de u et v prennent systématiquement deux valeurs consécutives de la suite de Fibonacci, car $F_{k+2} \bmod F_{k+1} = F_k$. Arrivé à $k = 1$, c'est-à-dire $u = 2$ et $v = 1$, on fait une dernière itération vers $u = 1$ et $v = 0$.

On déduit que le calcul de $\gcd(u, v)$ est en $O(\log u)$ car les nombres de Fibonacci ont une croissance exponentielle [à savoir $F_n \sim \frac{1}{\sqrt{5}}\varphi^n$ où $\varphi = (1 + \sqrt{5})/2$ est le nombre d'or].

Si on applique la fonction $\gcd$ à des entiers négatifs, on peut obtenir un résultat négatif.

En effet, l'opération modulo de C et d'OCaml, qui est celle de notre machine, renvoie une valeur du même signe que son premier argument.

Pour calculer le pgcd de deux entiers, le plus simple est donc de commencer par s'assurer qu'ils sont bien positifs ou nuls avant de lancer l'algorithme d'Euclide.

1.2 Algorithme d'Euclide étendu

On peut facilement modifier l'algorithme d'Euclide pour qu'il calcule également les coefficients de Bézout, c'est-à-dire deux entiers x et y tels que

$$xu + yv = \gcd(u, v). \quad (5)$$

Le programme suivante contient une fonction OCaml `extended_gcd` qui reçoit u et v en arguments et renvoie le triplet $(x, y, \gcd(u, v))$.

Algorithme d'Euclide étendu

Calcul du pgcd de deux entiers avec coefficients de Bézout.

— En OCaml, avec une fonction récursive :

```
let rec extended_gcd (u: int) (v: int) : int * int * int =
  if v = 0 then
    (1, 0, u)
  else
    let (x, y, d) = extended_gcd v (u mod v) in
    (y, x - y*(u/v), d)
```

— En C, avec une boucle :

```
int arith_extended_gcd(int u, int v, int *x, int *y) {
  int a = 1, b = 0, c = 0, d = 1;
  // invariant : u = a*u0 + b*v0 et v = c*u0 + d*v0
  while (v != 0) {
    int q = u / v, r = u % v;
    u = v; v = r;
    int olda = a, oldb = b;
    a = c; b = d;
    c = olda - q * c; d = oldb - q * d;
  }
  *x = a; *y = b;
  return u;
}
```

La fonction C renvoie le pgcd et stocke dans `*x` et `*y` les coefficients de Bézout. Dans l'invariant de boucle, u_0 et v_0 désignent les valeurs initiales de u et v .

On se convainc facilement que la troisième composante du triplet renvoyé est bien le pgcd de u et v . En effet, si on se focalise sur le calcul de cette composante uniquement, on retrouve exactement les mêmes calculs que ceux effectués par la fonction `gcd`.

Pour justifier la correction de l'algorithme d'Euclide étendu, on montre que le résultat renvoyé vérifie bien (5) par récurrence sur le nombre d'appels.

Dans le cas $v = 0$, c'est immédiat.

Sinon, l'appel récursif nous donne des valeurs correctes par hypothèse de récurrence, c'est-à-dire

$$xv + y(u \bmod v) = \gcd(v, u \bmod v)$$

Or, on a d'une part $\gcd(u, v) = \gcd(v, u \bmod v)$ et d'autre part $u \bmod v = u - v \lfloor u/v \rfloor$, ce qui donne bien, après un peu d'algèbre, l'identité recherchée

$$yu + (x - y \lfloor u/v \rfloor)v = \gcd(u, v).$$

Le programme contient également une fonction `C` qui réalise l'algorithme d'Euclide étendu, cette fois avec une boucle.

La complexité reste la même que pour la fonction `gcd`. Le nombre d'opérations effectuées à chaque tour de boucle est certes supérieur, mais il reste borné et le nombre d'itérations est exactement le même. La complexité est donc toujours $O(\log(\max(u, v)))$.

L'algorithme d'Euclide étendu peut notamment être utilisé pour calculer une division modulo m , ce que nous allons faire dans la section suivante.

2 Arithmétique modulaire

Par arithmétique modulaire, on entend l'idée de calculer modulo un certain entier m , c'est-à-dire de calculer dans $\mathbb{Z}/m\mathbb{Z}$. En particulier, on utilise alors la propriété que l'addition, la soustraction et la multiplication commutent avec le modulo,

$$\begin{aligned}(x + y) \bmod m &= ((x \bmod m) + (y \bmod m)) \bmod m, \\(x - y) \bmod m &= ((x \bmod m) - (y \bmod m)) \bmod m, \\(x \times y) \bmod m &= ((x \bmod m) \times (y \bmod m)) \bmod m.\end{aligned}$$

2.1 Calcul de x^n modulo m

On a déjà étudié l'algorithme d'exponentiation.

Pour calculer $x^n \bmod m$, il suffit d'utiliser exactement le même code, en ajoutant uniquement l'opération `% m` sur chaque calcul intermédiaire.

Le programme suivante en donne une version en C.

Calcul de x^n modulo m

```
uint64_t arith_power_mod(uint64_t x, uint64_t n, uint64_t m) {
    uint64_t y = 1;
    while (n > 0) {
        if (n % 2 == 1) y = (y * x) % m;
        x = (x * x) % m;
        n = n / 2;
    }
    return y;
}
```

Il est important d'appliquer l'opération modulo à chaque étape. Sans cela, les débordements arithmétiques vont conduire à un résultat incorrect.

Exemple

Supposons par exemple que l'on cherche à calculer $42^{1000} \bmod 10^9$. Si on fait le calcul sans appliquer de modulo systématiquement, avec des entiers 32 ou 64 bits, on obtiendra 0, car 2^{64} divise clairement 42^{1000} . Avec le programme, on obtient le bon résultat, à savoir 316 389 376.

Il y a encore une petite subtilité. Il faut pouvoir calculer le produit de deux entiers sans débordement arithmétique. Par conséquent, en calculant avec des entiers 64 bits comme dans le programme, on est de fait limité à des valeurs de m ne dépassant pas 2^{32} .

Si on calculait $42^{1000} \bmod 10^9$ avec seulement des entiers 32 bits, y compris en faisant une opération modulo à chaque étape, on obtiendrait un résultat incorrect (à savoir 330 742 272), quand bien même $10^9 < 2^{32}$. L'entier n , en revanche, peut être arbitrairement grand (dans la limite de $2^{64} - 1$, bien entendu), car on ne fait que le diviser par deux à chaque étape.

2.2 Calcul de l'inverse modulo m

L'algorithme d'Euclide étendu permet en particulier de calculer un *inverse modulaire* de x , c'est-à-dire un entier y tel que $xy \equiv 1 \pmod{m}$.

Un tel inverse existe si et seulement si x est premier avec m .

L'inverse modulaire est notamment utilisé dans l'algorithme de chiffrement asymétrique RSA.

Sous l'hypothèse $\gcd(x, m) = 1$, l'algorithme d'Euclide étendu nous donne deux entiers y et z tels que

$$yx + zm = \gcd(x, m) = 1.$$

Dès lors, $yx \equiv 1 \pmod{m}$ et y est donc l'inverse de x modulo m .

Le programme suivante contient une fonction `arith_inv_mod` qui réalise ce calcul.

Inverse de x modulo m

```
uint64_t arith_inv_mod(uint64_t x, uint64_t m) {
    int y, z;
    int g = arith_extended_gcd(x, m, &y, &z);
    assert(g == 1); // et 1 = y*x + z*m
    if (y < 0) y += m;
    return y;
}
```

L'algorithme d'Euclide peut renvoyer une valeur négative pour y (d'où le type `int`) mais sa valeur absolue ne peut dépasser m . L'avant-dernière ligne du code est là pour assurer que le résultat est bien dans $[0, m - 1]$.

Une fois que l'on sait calculer l'inverse modulaire, on sait diviser modulo m : pour diviser u par v , sous l'hypothèse que v est premier avec m , il suffit de multiplier u par l'inverse modulaire de v .

2.3 Crible d'Ératosthène

De nombreuses applications requièrent un test de primalité (l'entier n est-il premier ?) ou le calcul exhaustif des nombres premiers jusqu'à un certain rang.

Le crible d'Ératosthène est un algorithme qui détermine, pour un certain entier N , la primalité de tous les entiers $n \leq N$.

Exemple

Illustrons son fonctionnement avec $N = 23$. On écrit tous les entiers de 0 à N .

On va éliminer progressivement tous les entiers qui ne sont pas premiers — d'où le nom de crible.

Initialement, on se contente de dire que 0 et 1 ne sont pas premiers.

Puis on détermine le premier entier non encore éliminé. Il s'agit de 2. On élimine alors tous ses multiples, à savoir ici tous les entiers pairs supérieurs à 2.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Puis on recommence. Le prochain entier non éliminé est 3. On élimine donc à leur tour tous les multiples de 3.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

On note que certains étaient déjà éliminés (les multiples de 6, en l'occurrence) mais ce n'est pas grave. Le prochain entier non éliminé est 5. Comme $5 \times 5 > 23$ le crible est terminé. En effet, tout multiple de 5, c'est-à-dire $k \times 5$, est soit déjà éliminé si $k < 5$, soit au-delà de 23 si $k \geq 5$. Les nombres premiers inférieurs ou égaux à N sont alors tous les nombres qui n'ont pas été éliminés, c'est-à-dire ici 2, 3, 5, 7, 11, 13, 17, 19 et 23.

Le programme suivante contient une fonction C qui réalise le crible d'Ératosthène, pour tous les entiers jusqu'à `max` inclus.

Crible d'Ératosthène

```

1 bool *sieve(int max) {
2     bool *prime = calloc(max + 1, sizeof(bool));
3     for (int i = 2; i <= max; i++)
4         prime[i] = true;
5     for (int n = 2; n * n <= max; n++)
6         if (prime[n])
7             for (int m = n * n; m <= max; m += n)
8                 prime[m] = false;
9     return prime;
10 }
```

Elle alloue un tableau de booléens (ligne 2), qui sera renvoyé à la fin.

Le tableau contient initialement la valeur `false` dans toutes ces cases (`calloc` initialise la zone mémoire avec la valeur 0) et on commence donc par écrire `true` dans toutes les cases d'indice $i \geq 2$ (lignes 3–4). La boucle principale du crible parcourt alors les entiers de 2 à

$\lfloor \sqrt{\text{max}} \rfloor$ (ligne 5) et teste à chaque fois leur primalité (ligne 6). Le cas échéant, elle élimine les multiples de n , à l'aide d'une seconde boucle (lignes 7–8). La même raison qui nous permet d'arrêter le crible dès que $n \times n > \text{max}$ nous permet de démarrer cette élimination à $n \times n$ (plutôt que $2n$), les multiples plus petits ayant déjà été éliminés. Il ne reste plus qu'à renvoyer le tableau de booléens (ligne 9).

Évaluons la complexité du crible d'Ératosthène.

- La complexité en espace est clairement $O(N)$.
- La complexité en temps nous amène à considérer le coût de chaque itération de la boucle principale. S'il ne s'agit pas d'un nombre premier, le coût est constant (on ne fait rien). Mais lorsqu'il s'agit d'un nombre premier p , alors la boucle interne a un coût $\frac{N}{p}$ car on considère tous les multiples de p (en fait, un peu moins car on commence l'itération à p^2 , mais cela ne change pas l'asymptotique). Le coût total est donc

$$N + \sum_{p \leq N} \frac{N}{p}.$$

où la somme est faite sur les nombres premiers.
Mais comme un théorème d'Euler nous dit que

$$\sum_{p \leq N} \frac{1}{p} \sim \ln(\ln N),$$

d'où une complexité $N \ln(\ln N)$ pour le crible d'Ératosthène.

3 Retour sur trace (*backtracking*)

La technique du retour sur trace (en anglais *backtracking*) consiste à construire la solution d'un problème incrémentalement, en s'interrompant dès que l'on peut déterminer à coup sûr qu'une solution partielle ne pourra être complétée. Dès lors, on rebrousse chemin pour changer l'une des décisions prises précédemment. On peut s'arrêter dès qu'une solution est trouvée ou énumérer toutes les solutions. Le retour sur trace est une technique que chacun a déjà utilisée empiriquement, pour résoudre des jeux logiques tels que les mots croisés par exemple. La technique du retour sur trace est parfois appelée également *rebroussement* dans la littérature.

En termes de programmation, le retour sur trace est plus une méthode générale qu'un algorithme. Sa mise en œuvre va être différente pour chaque problème, les constantes étant seulement les idées générales d'exploration systématique et d'interruption prématurée.

On illustre le principe du retour sur trace sur un problème très classique, à savoir le Sudoku.


```
bool same_zone(int c1, int c2) {
    return row(c1) == row(c2)
        || col(c1) == col(c2)
        || group(c1) == group(c2);
}
```

Pour interrompre prématurément notre recherche d'une solution, on écrit une fonction booléenne `check` qui vérifie si la case `p` contient une valeur en conflit avec une autre case. Pour cela, on parcourt toutes les cases et, pour chacune, on teste l'existence d'un conflit avec la case `p`. Le cas échéant, on le signale immédiatement.

```
bool check(int grid[81], int p) {
    for (int c = 0; c < 81; c++)
        if (c != p && same_zone(p, c) && grid[p] == grid[c])
            return false;
    return true;
}
```

Munis de cette fonction `check`, on peut maintenant écrire le code de la fonction `solve` qui réalise l'algorithme de retour sur trace. Elle commence par une recherche de la première case vide de la grille, par un simple parcours de toutes les cases.

```
bool solve(int grid[81]) {
    for (int c = 0; c < 81; c++)
        if (grid[c] == 0) {
```

Pour cette case, on va essayer successivement toutes les valeurs v possibles.

```
        for (int v = 1; v <= 9; v++) {
            grid[c] = v;
```

La valeur v étant affectée à la case `c`, on teste l'absence de conflit avec la fonction `check`. Le cas échéant, on rappelle `solve` récursivement pour continuer la résolution du problème. Si `solve` trouve une solution, on termine immédiatement en signalant le succès de la recherche.

```
            if (check(grid, c) && solve(grid))
                return true;
```

On note que la fonction `solve` n'est pas appelée si `check` renvoie `false`, car l'opérateur `&&` est paresseux. Si en revanche on sort de la boucle `for`, c'est que les 9 valeurs possibles ont toutes été essayées sans succès. Dans ce cas, on restaure la valeur 0 dans la case `c`, puis on signale l'échec de la recherche.

```
        }
        grid[c] = 0;
        return false;
    }
```

Il est important de comprendre que seule la première case vide a été considérée. En effet, si une solution existe, alors la valeur correspondante de la case `c` aurait dû mener à cette solution. Il est donc inutile de considérer les autres cases vides. Ce serait une perte de temps considérable. Si en revanche on sort de la boucle `for`, c'est que la grille ne contient aucune case vide. Vu qu'on a supposé la grille sans contradiction en entrée, c'est donc une solution et on signale le succès.

```
return true;
```

Le code complet est donné dans le programme suivante.

Résolution du Sudoku

```
int row(int c) { return c / 9; }
int col(int c) { return c % 9; }
int group(int c) { return 3 * (row(c) / 3) + col(c) / 3; }

bool same_zone(int c1, int c2) {
    return row(c1) == row(c2)
        || col(c1) == col(c2)
        || group(c1) == group(c2);
}

// vérifie que la valeur à la position p est compatible avec les autres
bool check(int grid[81], int p) {
    for (int c = 0; c < 81; c++)
        if (c != p && same_zone(p, c) && grid[p] == grid[c])
            return false;
    return true;
}

// algorithme de retour sur trace (backtracking)
// entrée:
// - grid ne contient pas de contradiction
// sortie:
// - true si grid a pu être complétée en une solution
// - false si ce n'est pas possible *et* grid est inchangée
bool solve(int grid[81]) {
    for (int c = 0; c < 81; c++)
        if (grid[c] == 0) {
            for (int v = 1; v <= 9; v++) {
                grid[c] = v;
                if (check(grid, c) && solve(grid))
                    return true;
            }
            grid[c] = 0;
            return false;
        }
    return true;
}
```

Un tel code résout notre problème de Sudoku en un dixième de seconde. On pourrait penser qu'on a été particulièrement chanceux, mais un test plus poussé sur 243 problèmes de Sudoku nous montre que ce n'est pas le cas. Il faut moins de 7 secondes pour résoudre tous ces problèmes, soit moins d'un tiers de seconde par problème.

Il n'est pas facile d'analyser ou prédire ce qui se passe exactement dans un tel programme. On peut tout de même faire quelques observations empiriques.

Une instrumentation facile de notre code montre qu'on a fait exactement 142 256 appels à `solve`. C'est notamment très peu au regard de l'espace potentiel de recherche.

En effet, on est partis d'une grille avec 58 cases vides, avec 9 valeurs pour chacune, ce qui correspond à un espace de recherche de taille $9^{58} \approx 2,22 \times 10^{55}$. Même en prenant en compte les contraintes initiales, qui limitent les valeurs possibles pour chaque case vide, on aurait encore

$$4 \times 4 \times 3 \times \dots \times 3 \approx 1,42 \times 10^{32}$$

ensembles de valeurs dans l'espace de recherche.

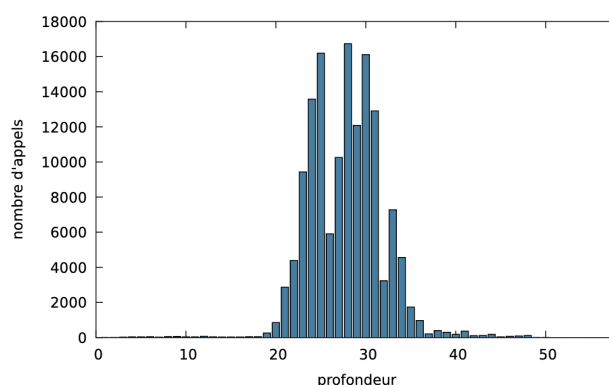
On comprend en particulier qu'une recherche brutale, testant systématiquement chaque n -uplet de valeurs possibles pour les cases vides, n'a aucune chance de terminer dans un délai raisonnable. Ce qui fait la force de notre algorithme, c'est l'*interruption prématurée* de la recherche dès lors que la valeur placée est incompatible avec les valeurs déjà présentes, sans chercher à donner des valeurs aux autres cases.

On le comprend mieux encore si on observe comment se répartissent tous les appels à `solve` à chaque profondeur dans la pile d'appels. Ici, la profondeur varie entre 0 pour le tout premier appel et 58 pour un appel sur une grille pleine.

Là encore, une instrumentation simple de notre code nous donne cette information :

profondeur	nombre d'appels
0	1
1	4
2	11
3	28
$\vdots$	

En particulier, on constate que notre programme a essayé successivement quatre valeurs pour la première case libre (la case (0,1) en l'occurrence) avant de trouver une valeur permettant une solution (la valeur 7 en l'occurrence). La figure suivante trace l'histogramme de ces profondeurs.



Comme on le constate, il y a peu d'appels à des profondeurs faibles, car la combinatoire est encore peu importante, et également peu d'appels à des profondeurs élevées, car plus la grille est remplie et plus la fonction `check` renvoie la valeur `false`. En revanche, dans les profondeurs intermédiaires, entre 20 et 40, on a de très nombreux appels à `solve`. Cette distribution est caractéristique des algorithmes de retour sur trace.

Il est important de noter que notre programme occupe *très peu de mémoire*, même s'il devait tourner très longtemps et explorer un très grand espace de recherche. Il n'y a en particulier aucune allocation sur le tas. Et par ailleurs, il n'y a aucun risque de faire déborder la pile d'appels car il n'y a jamais plus de 82 appels imbriqués.

Il est facile d'adapter notre programme pour trouver *toutes les solutions* du problème. On trouve ici que notre grille n'a en fait qu'une seule solution, et cela prend à peine plus de temps (124 ms) et à peine plus d'appels à `solve` (148 983). Sur une grille moins contrainte, cependant, il pourrait y avoir de nombreuses solutions et le temps de calcul pourrait devenir beaucoup plus important.

Enfin, il convient de préciser que pour des problèmes de Sudoku plus grands (16×16 , 25×25 , etc.), il deviendrait nécessaire d'améliorer l'efficacité de notre programme dans sa recherche des cases vides et dans son exploration des valeurs possibles.

Maintenir par exemple en permanence l'ensemble des valeurs possibles pour chaque case est une bonne approche. Pour autant, le principe du retour sur trace resterait exactement le même.

Les applications du retour sur trace sont innombrables. L'exercice 146 page 596 propose d'appliquer le retour sur trace à un autre problème très classique, à savoir le problème des N reines.

Un algorithme pour les unifier tous ?

Pour chaque problème pour lequel s'applique la technique du retour sur trace, on peut écrire un programme spécifique pour le résoudre. Mais y a-t-il un problème auquel on puisse se ramener systématiquement, ou du moins souvent, dans l'espoir d'écrire un programme une fois pour toutes ? La réponse est oui, avec le problème de la *couverture exacte* (en anglais *exact cover*). Étant donnée une matrice de 0 et de 1, comme par exemple cette matrice

$$\begin{pmatrix} 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{pmatrix}$$

existe-t-il un sous-ensemble de lignes couvrant chaque colonne exactement une fois ?

En 2000, Donald Knuth a proposé un algorithme très efficace pour résoudre le problème de la couverture exacte, connu sous le nom de *liens dansants*. Il élimine progressivement des colonnes et des lignes de la matrice et procède par rebroussement en cas d'échec.